

TASK MANAGEMENT SYSTEM

Project Documentation

Technology Stack: Java EE · JSP · Servlets · MySQL · Apache Tomcat 10.5

Developer: Divy Bhanushali

Device: DIVY | Academic Year: 2024–2025

1. Project Name

Task Management System

A role-based web application enabling managers and employees to create, assign, track, update, and delete tasks through a browser-based interface — built entirely with Java EE, JSP, and MySQL.

2. Abstract

The Task Management System is a web-based application developed using Java EE technologies — Java Servlets, JavaServer Pages (JSP), and MySQL as the backend database. The system provides a structured platform for task delegation and tracking within an organisation.

Two user roles are supported — Manager and Employee. A manager can view all tasks across the system, mark them as completed, or delete them. An employee can add new tasks and monitor their own task statuses. Authentication is handled by a login module that validates credentials stored in the MySQL database and redirects users to their respective role-specific dashboards.

The project follows the MVC (Model-View-Controller) architectural pattern: model classes (Task, User) represent data; DAO classes (TaskDAO, UserDAO) encapsulate all SQL operations; Servlet classes act as controllers; and JSP pages serve as views. The application is deployed on Apache Tomcat / TomEE 10.5 using MySQL Connector/J 9.6.0 for JDBC connectivity.

This system addresses the common need for lightweight task tracking in small to mid-sized teams — providing a clean, accessible interface without relying on complex third-party tools or frameworks.

3. Hardware Requirements

3.1 Development Machine (Used for this project)

Component	Specification
Device Name	DIVY
Processor	12th Gen Intel Core i5-12450H (2.00 GHz, up to 4.40 GHz)
Architecture	x64-based, 64-bit operating system
Installed RAM	8.00 GB (7.65 GB usable)
System Type	64-bit Operating System
Storage	512 GB SSD (project stored locally)
Display	1920 x 1080 Full HD

3.2 Minimum Hardware to Run the Application

Component	Minimum Specification
Processor	Intel Core i3 or equivalent (1 GHz+)

RAM	4 GB minimum (8 GB recommended)
Hard Disk	20 GB free disk space
Display	1280 x 720 resolution or higher
Network	Localhost (127.0.0.1) or LAN
Input Devices	Keyboard and Mouse

4. Software Requirements

Software	Version	Purpose
Java (JDK)	JDK 17 or above	Core language & runtime
Apache Tomcat / TomEE	10.5	Servlet container / web server
MySQL Server	8.x	Relational database
MySQL Connector/J	9.6.0 (mysql-connector-j-9.6.0.jar)	JDBC driver for MySQL
JSON Simple	1.1.1 (json-simple-1.1.1.jar)	JSON parsing utility
IDE	Eclipse / IntelliJ IDEA / NetBeans	Development environment
Web Browser	Chrome / Firefox / Edge	Front-end client
Operating System	Windows 11 (64-bit)	Host OS on DIVY machine
MySQL Workbench	8.x	Database GUI & testing

5. Model / Architecture Used

5.1 MVC (Model-View-Controller) Pattern

The application follows the MVC architectural pattern to maintain separation of concerns:

- **Model** — User.java and Task.java: Plain Java POJOs representing database entities.
- **View** — JSP Pages: login.jsp, register.jsp, employeeDashboard.jsp, managerDashboard.jsp render the HTML UI.
- **Controller** — Servlets: LoginServlet, RegisterServlet, TaskServlet, ManagerServlet process HTTP requests and route responses.
- **DAO Layer** — TaskDAO.java and UserDAO.java: Encapsulate all SQL queries, keeping database logic separate from business logic.

5.2 Three-Tier Architecture

Tier	Technology	Role
Presentation Layer	JSP + CSS	User Interface displayed in browser
Business Logic Layer	Java Servlets + DAOs	Processing requests, routing responses
Data Layer	MySQL via JDBC	Persistent storage of users & tasks

5.3 Database Schema (taskdb)

Table: users

Column	Data Type	Description
user_id	INT (PK, AUTO_INCREMENT)	Unique user identifier
name	VARCHAR(100)	Full name of the user
email	VARCHAR(100) UNIQUE	Login email address
password	VARCHAR(100)	User password
role	ENUM('manager','employee')	User role

Table: tasks

Column	Data Type	Description
task_id	INT (PK, AUTO_INCREMENT)	Unique task identifier
title	VARCHAR(200)	Task title
description	TEXT	Task description
status	ENUM('pending','completed')	Current status, default=pending

employee_id	INT (FK → users.user_id)	Assigned employee reference
-------------	--------------------------	-----------------------------

SQL to create the database:

```
CREATE DATABASE taskdb;
USE taskdb;
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE,
    password VARCHAR(100),
    role ENUM('manager','employee')
);
CREATE TABLE tasks (
    task_id INT AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200),
    description TEXT,
    status ENUM('pending','completed') DEFAULT 'pending',
    employee_id INT,
    FOREIGN KEY (employee_id) REFERENCES users(user_id)
);
```

6. Testing Methods Used

6.1 Black-Box / Manual Testing

Each feature was tested end-to-end by interacting with the live browser interface and comparing outputs against expected behaviour.

#	Test Case	Steps / Input	Expected Output	Result
1	Valid Login — Employee	harshit@gmail.com + correct password	Redirect to employeeDashboard.jsp	PASS
2	Valid Login — Manager	divy@gmail.com + correct password	Redirect to managerDashboard.jsp	PASS
3	Invalid Login	Wrong email or password	login.jsp?error=1 shown	PASS
4	Register — Manager	divy@gmail.com, role=Manager	Row inserted in users table	PASS
5	Register — Employee	harshit@gmail.com, role=Employee	Row inserted in users table	PASS
6	Add Task	'Fix Login Page', submit	Task saved with status=pending	PASS
7	View Own Tasks	Employee opens dashboard	Fix Login Page + Add CSS listed	PASS
8	View All Tasks — Manager	Manager opens dashboard	All 3 tasks from all employees shown	PASS
9	Mark Task Complete	Manager clicks Complete	status updated to 'completed'	PASS
10	Delete Task	Manager clicks Delete	Task removed from DB and list	PASS
11	Session Protection	Access dashboard URL without login	Redirect to login.jsp	PASS
12	Logout	Click Logout button	Session invalidated, redirect login	PASS

6.2 Unit-Level Testing

Individual DAO methods were verified by inserting test data and inspecting results in MySQL Workbench:

- TaskDAO.addTask() — verified row insertion with status = pending.
- TaskDAO.getTasks(empId) — verified filtered fetch per employee_id.
- TaskDAO.getAllTasks() — verified all task records returned.
- TaskDAO.updateStatus(id) — verified status changed to 'completed'.
- TaskDAO.deleteTask(id) — verified row removed from tasks table.
- UserDAO.login() — tested valid credentials (returned User object) and invalid (returned null).

6.3 Integration Testing

Servlet-to-DAO-to-database flow was validated by triggering actions from the browser and immediately checking the resulting state in MySQL Workbench after each operation.

6.4 Boundary & Negative Testing

- Empty login fields — blocked by HTML5 required attribute before reaching the servlet.
- Empty task title/description — blocked by required attribute on the form.
- Direct URL access to dashboards without an active session — correctly redirected to login.jsp.

7. Source Code

7.1 DBConnection.java

```
package dao;
import java.sql.*;
public class DBConnection {
    private static final String URL = "jdbc:mysql://localhost:3306/taskdb";
    private static final String USER = "root";
    private static final String PASS = "system";
    public static Connection getConnection() throws Exception {
        Class.forName("com.mysql.cj.jdbc.Driver");
        return DriverManager.getConnection(URL, USER, PASS);
    }
}
```

7.2 User.java

```
package model;
public class User {
    private int id;
    private String name, email, password, role;
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }
    public String getPassword() { return password; }
    public void setPassword(String password) { this.password = password; }
    public String getRole() { return role; }
    public void setRole(String role) { this.role = role; }
}
```

7.3 Task.java

```
package model;
public class Task {
    private int id;
    private String title, description, status;
    private int employeeId;
    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getTitle() { return title; }
    public void setTitle(String title) { this.title = title; }
    public String getDescription() { return description; }
    public void setDescription(String d) { this.description = d; }
    public String getStatus() { return status; }
    public void setStatus(String status) { this.status = status; }
    public int getEmployeeId() { return employeeId; }
    public void setEmployeeId(int id) { this.employeeId = id; }
}
```

7.4 UserDAO.java

```
package dao;
import java.sql.*;
import model.User;
public class UserDAO {
    public User login(String email, String password) {
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM users WHERE email=? AND password=?");
            ps.setString(1, email);
            ps.setString(2, password);
            ResultSet rs = ps.executeQuery();
            if (rs.next()) {
                User u = new User();
                u.setId(rs.getInt("user_id"));
                u.setName(rs.getString("name"));
                u.setRole(rs.getString("role"));
                return u;
            }
        } catch (Exception e) { e.printStackTrace(); }
        return null;
    }
}
```

7.5 TaskDAO.java

```
package dao;
import java.sql.*;
import java.util.*;
import model.Task;
public class TaskDAO {
    public void addTask(Task t) {
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO tasks(title,description,status,employee_id) VALUES
(?,?,?,?)");
            ps.setString(1, t.getTitle());
            ps.setString(2, t.getDescription());
            ps.setString(3, "pending");
            ps.setInt(4, t.getEmployeeId());
            ps.executeUpdate();
        } catch (Exception e) { e.printStackTrace(); }
    }
    public List<Task> getTasks(int empId) {
        List<Task> list = new ArrayList<>();
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "SELECT * FROM tasks WHERE employee_id=?");
            ps.setInt(1, empId);
            ResultSet rs = ps.executeQuery();
            while (rs.next()) {
                Task t = new Task();
            }
        }
    }
}
```

```

        t.setId(rs.getInt("task_id"));
        t.setTitle(rs.getString("title"));
        t.setDescription(rs.getString("description"));
        t.setStatus(rs.getString("status"));
        list.add(t);
    }
} catch (Exception e) { e.printStackTrace(); }
return list;
}

public List<Task> getAllTasks() {
    List<Task> list = new ArrayList<>();
    try (Connection con = DBConnection.getConnection()) {
        ResultSet rs = con.createStatement().executeQuery("SELECT * FROM
tasks");
        while (rs.next()) {
            Task t = new Task();
            t.setId(rs.getInt("task_id"));
            t.setTitle(rs.getString("title"));
            t.setDescription(rs.getString("description"));
            t.setStatus(rs.getString("status"));
            list.add(t);
        }
    } catch (Exception e) { e.printStackTrace(); }
    return list;
}

public void updateStatus(int id) {
    try (Connection con = DBConnection.getConnection()) {
        PreparedStatement ps = con.prepareStatement(
            "UPDATE tasks SET status='completed' WHERE task_id=?");
        ps.setInt(1, id);
        ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
}

public void deleteTask(int id) {
    try (Connection con = DBConnection.getConnection()) {
        PreparedStatement ps = con.prepareStatement(
            "DELETE FROM tasks WHERE task_id=?");
        ps.setInt(1, id);
        ps.executeUpdate();
    } catch (Exception e) { e.printStackTrace(); }
}
}

```

7.6 LoginServlet.java

```

package servlet;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;
import dao.UserDAO;
import model.User;
public class LoginServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {

```

```

String email = req.getParameter("email");
String pass  = req.getParameter("password");
UserDAO dao = new UserDAO();
User u = dao.login(email, pass);
if (u != null) {
    HttpSession session = req.getSession();
    session.setAttribute("user", u);
    if (u.getRole().equals("manager"))
        res.sendRedirect("managerDashboard.jsp");
    else
        res.sendRedirect("employeeDashboard.jsp");
} else {
    res.sendRedirect("login.jsp?error=1");
}
}
}

```

7.7 RegisterServlet.java

```

package servlet;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;
import java.sql.*;
import dao.DBConnection;
public class RegisterServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String name  = req.getParameter("name");
        String email = req.getParameter("email");
        String pass  = req.getParameter("password");
        String role  = req.getParameter("role");
        try (Connection con = DBConnection.getConnection()) {
            PreparedStatement ps = con.prepareStatement(
                "INSERT INTO users(name,email,password,role) VALUES (?, ?, ?, ?)");
            ps.setString(1, name);
            ps.setString(2, email);
            ps.setString(3, pass);
            ps.setString(4, role);
            ps.executeUpdate();
            res.sendRedirect("login.jsp");
        } catch (Exception e) { e.printStackTrace(); }
    }
}

```

7.8 TaskServlet.java

```

package servlet;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;
import dao.TaskDAO;

```

```
import model.Task;
public class TaskServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String action = req.getParameter("action");
        TaskDAO dao = new TaskDAO();
        if (action.equals("add")) {
            Task t = new Task();
            t.setTitle(req.getParameter("title"));
            t.setDescription(req.getParameter("desc"));
            t.setEmployeeId(Integer.parseInt(req.getParameter("empId")));
            dao.addTask(t);
        }
        res.sendRedirect("employeeDashboard.jsp");
    }
}
```

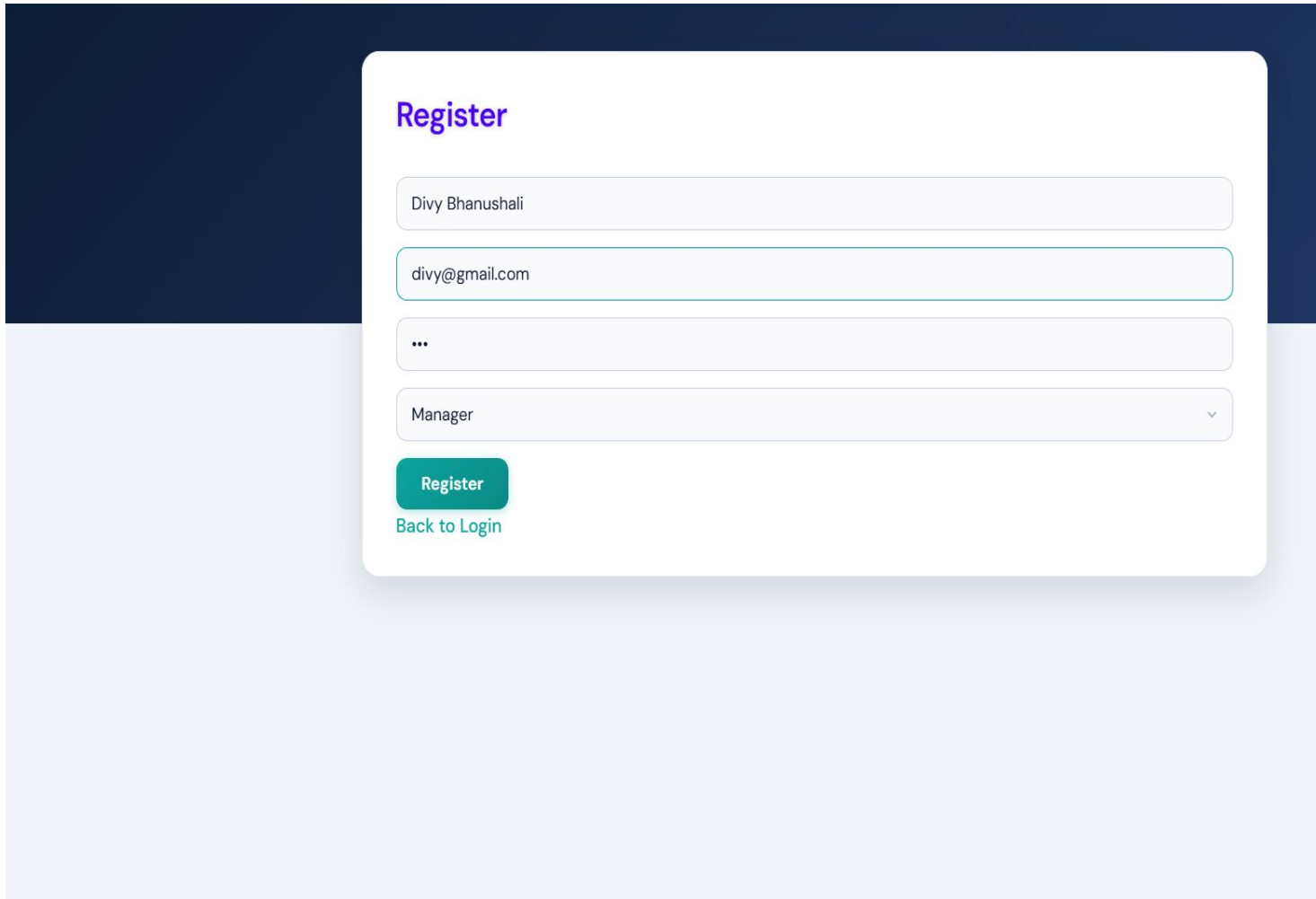
7.9 ManagerServlet.java

```
package servlet;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;
import dao.TaskDAO;
public class ManagerServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException {
        String action = req.getParameter("action");
        int id = Integer.parseInt(req.getParameter("id"));
        TaskDAO dao = new TaskDAO();
        if (action.equals("complete")) dao.updateStatus(id);
        else if (action.equals("delete")) dao.deleteTask(id);
        res.sendRedirect("managerDashboard.jsp");
    }
}
```

8. Screenshots

The following screenshots were captured from the live running application on device DIVY during testing.

8.1 Register Page — Manager (Divy Bhanushali)



The screenshot displays a registration form titled "Register" in purple text. The form is set against a dark blue header and a light blue background. It contains four input fields: "Name" with the value "Divy Bhanushali", "Email" with "divy@gmail.com", "Password" with three dots, and "Role" with "Manager" selected from a dropdown menu. Below the fields is a green "Register" button and a blue "Back to Login" link.

Figure 8.1 — Registration page with Manager role selected

8.2 Login Page



The image shows a login form on a dark blue background. The form is white with rounded corners and a subtle shadow. It features a purple 'Login' heading, a light blue email input field containing 'divy@gmail.com', a white password input field with a green border and a masked password '...|', a teal 'Login' button, and a teal 'New User? Register' button.

Login

divy@gmail.com

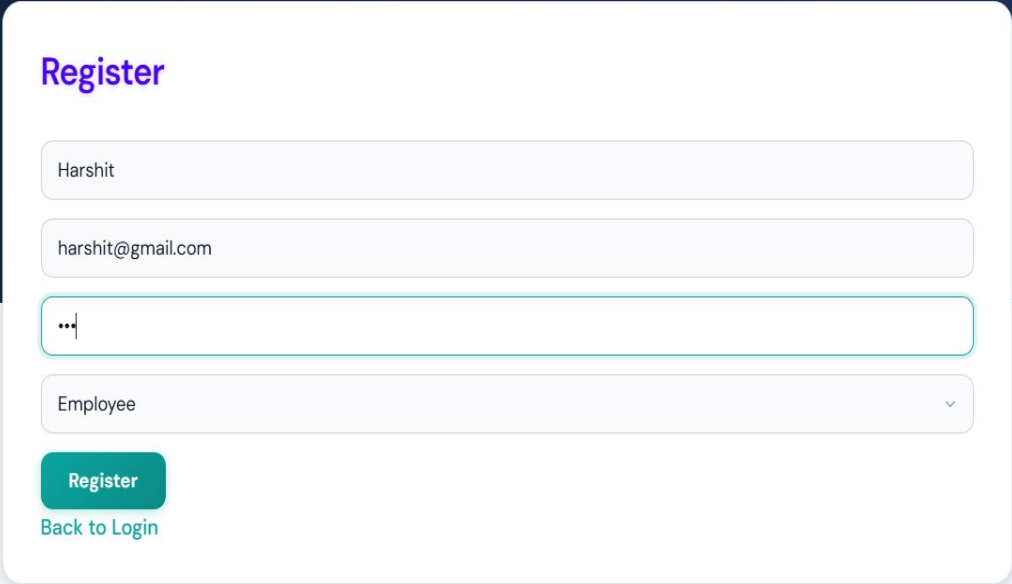
...|

Login

New User? Register

Figure 8.2 — Login page with email and password fields

8.3 Register Page — Employee (Harshit)



The registration form is titled "Register" in purple. It contains four input fields: a text field for the name "Harshit", an email field for "harshit@gmail.com", a password field with a strength indicator (three dots) and a teal border, and a role dropdown menu currently set to "Employee". Below the fields is a teal "Register" button and a teal link "Back to Login".

Register

Harshit

harshit@gmail.com

...

Employee

Register

[Back to Login](#)

Figure 8.3 — Registration page with Employee role selected

8.4 Employee Dashboard (Harshit)

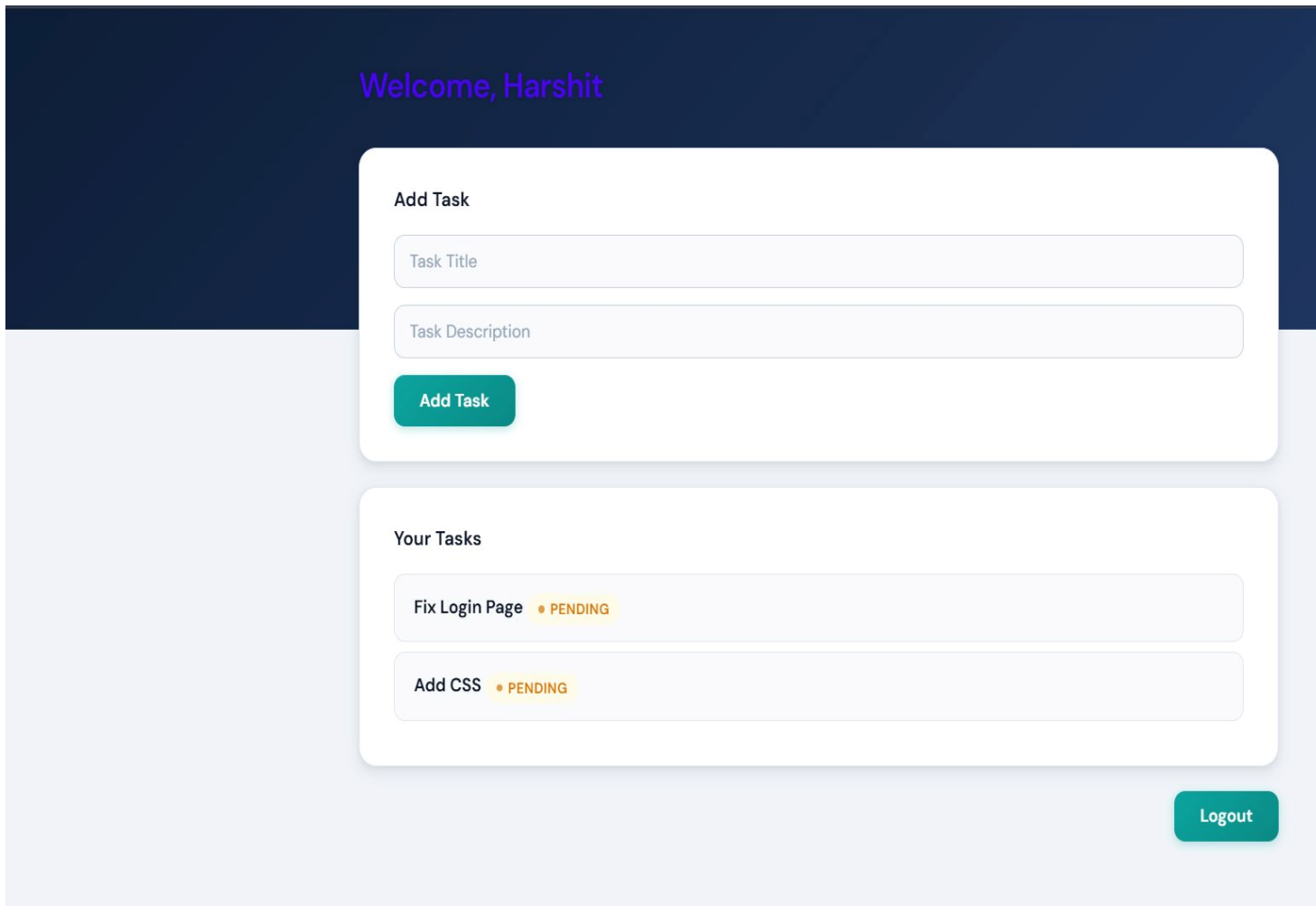


Figure 8.4 — Employee dashboard showing Add Task form and task list with PENDING status badges

8.5 Manager Dashboard (Divy)

Manager Dashboard

All Tasks

Fix Login Page • PENDING Complete Delete

Add CSS • PENDING Complete Delete

Check Payment Gateway Error • PENDING Complete Delete

Logout

Figure 8.5 — Manager dashboard showing all tasks with Complete and Delete action links

9. Gantt Chart

Project schedule for Task Management System — 6-week development plan:

#	Activity	Wk 1	Wk 2	Wk 3	Wk 4	Wk 5	Wk 6
1	Requirement Analysis						
2	System Design						
3	Database Design						
4	Backend Development						
5	Frontend (JSP/CSS)						
6	Integration & Testing						
7	Bug Fixing & Review						
8	Documentation						
9	Final Deployment						

Blue cells indicate active development phases for each activity.

10. Bibliography & References

1. Oracle Corporation. (2024). Java Servlet Technology — Java EE 7 Tutorial.
<https://docs.oracle.com/javaee/7/tutorial/servlets.htm>
2. Oracle Corporation. (2024). JavaServer Pages Technology — Java EE 7 Tutorial.
<https://docs.oracle.com/javaee/7/tutorial/jsf-page.htm>
3. Apache Software Foundation. (2024). Apache Tomcat 10.5 Documentation.
<https://tomcat.apache.org/tomcat-10.5-doc/>
4. MySQL AB / Oracle Corporation. (2024). MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc/refman/8.0/en/>
5. MySQL AB / Oracle Corporation. (2024). MySQL Connector/J 9.6 Developer Guide.
<https://dev.mysql.com/doc/connector-j/en/>
6. JSON.org. (2024). JSON Simple Library 1.1.1. <https://code.google.com/archive/p/json-simple/>
7. Deitel, P. & Deitel, H. (2020). Java How to Program (11th ed.). Pearson Education.
8. Murach, J. (2019). Murach's Java Servlets and JSP (3rd ed.). Mike Murach & Associates.
9. Horstmann, C. S. (2022). Core Java Volume II — Advanced Features (12th ed.). Pearson.
10. MDN Web Docs. (2024). HTML & CSS Reference. Mozilla Foundation.
<https://developer.mozilla.org/>

Appendix A — Project File Structure

```
TaskManagementSystem/  
  WEB-INF/  
    web.xml  
    lib/  
      mysql-connector-j-9.6.0.jar  
      json-simple-1.1.1.jar  
    classes/  
      model/  User.class  Task.class  
      dao/    DBConnection.class  UserDAO.class  TaskDAO.class  
      servlet/ LoginServlet.class  RegisterServlet.class  
                TaskServlet.class  ManagerServlet.class  
  login.jsp  register.jsp  
  employeeDashboard.jsp  managerDashboard.jsp  Logout.jsp  
  style.css
```

Appendix B — web.xml Servlet Mappings

```
<web-app xmlns="https://jakarta.ee/xml/ns/jakartaee" version="5.0">  
  <servlet><servlet-name>LoginServlet</servlet-name>  
    <servlet-class>servlet.LoginServlet</servlet-class></servlet>  
  <servlet-mapping><servlet-name>LoginServlet</servlet-name>  
    <url-pattern>/LoginServlet</url-pattern></servlet-mapping>  
  <!-- Same pattern for RegisterServlet, TaskServlet, ManagerServlet -->  
</web-app>
```